IBM Innovation Centre for Education

NCS Nihon Communication Solutions

Yenepoya Institute of Arts, Science, Commerce, and Management

Project – High Level Design

on

Cloud Forensics Automation for Retail Breaches

Student Name: Saibin K Siju

Industry Mentor: Mr. Shashank Korram **Guided by: Ms. Prathiksha Mudipu**

Table of Contents

1. Introduction....3
   1.1. Scope of the document....3
   1.2. Intended Audience....3
   1.3. System overview....4

2. System Design....4
   2.1. Application Design....4
   2.2. Process Flow....5
   2.3. Information Flow....6
   2.4. Components Design....7
   2.5. Key Design Considerations....8
   2.6. API Catalogue....9

3. Data Design....10
   3.1. Data Model....10
   3.2. Data Access Mechanism....11
   3.3. Data Retention Policies....12
   3.4. Data Migration....12

4. Interfaces....13

5. State and Session Management....14

6. Caching....14

7. Non-Functional Requirements....15
   7.1. Security Aspects....15
   7.2. Performance Aspects....16

8. References....17

1. Introduction

This document provides a High-Level Design (HLD) for the Cloud Forensics Automation for Retail Breaches project (PRJN26-172). It describes the overall architectural design, component interactions, data flows, and technical decisions that form the foundation of the forensic file evidence collection system.

The purpose of this document is to provide a blueprint for developers, evaluators, and academic reviewers to understand how the system is structured, how its components interact, and how it meets the requirements of a real-time retail breach forensics platform. This document complements the Low-Level Design (LLD) by providing a system-wide architectural perspective.

1.1 Scope of the Document

This HLD document covers the following aspects of the Cloud Forensics Automation system:

- The overall system architecture and two-tier design separating the forensic engine from the web interface.
- The design of the Flask-based backend and the HTML5/CSS3/JavaScript frontend.
- The file scanning, SHA256 hash comparison, and tamper detection pipeline.
- The PostgreSQL evidence storage design and CSV export mechanism.
- Interfaces between system components, API endpoints, and external dependencies.
- Non-functional requirements including security and performance considerations.

1.2 Intended Audience

This document is intended for the following readers:

- Academic evaluators and faculty — to evaluate the system architecture and design quality.
- The development team or student — to guide implementation and understand component responsibilities.
- Cybersecurity researchers — to study the forensic scanning methodology and evidence model.
- Project reviewers and industry mentors — to verify that the system meets the defined IBM project objectives.

1.3 System Overview

The Cloud Forensics Automation for Retail Breaches system is a web-based forensic evidence platform designed to monitor files in a retail environment's digital directories and detect unauthorized modifications. It simulates a real-world retail data breach investigation scenario.

The system works by continuously monitoring a designated scan_folder directory. Any time a file is added, modified, or deleted, the watchdog observer triggers an automatic scan. The scanner computes SHA256 cryptographic hashes and compares them against previously recorded values. Each file is classified as NEW, UNCHANGED, MODIFIED, or DELETED, and assigned a risk level of CRITICAL, HIGH, MEDIUM, or LOW. All results are stored in a PostgreSQL database and displayed in real time on a Flask-based admin dashboard.

2. System Design

2.1 Application Design

The application follows a two-tier architecture separating the forensic backend engine from the web interface. The system is built using Python and Flask as the core framework, with PostgreSQL as the evidence store and HTML5/CSS3/Vanilla JavaScript for the frontend.

The client side consists of client.html for the file upload portal, index.html for the admin forensics dashboard, edit.html for the file editor, and view.html for the file viewer — all rendered by Flask using the Jinja2 template engine. The backend is managed by app.py which handles all routing, API endpoints, and business logic.

When a file is uploaded or modified, the backend saves it to scan_folder and invokes scanner.py via subprocess. The scanner walks the directory, computes SHA256 hashes using hashlib, compares against previous hashes retrieved from PostgreSQL via db.py, assigns statuses, and inserts results back into the evidence table. The watcher.py module runs separately using the watchdog Observer to trigger scans in real time on any filesystem event.

The system uses Python 3.8+ as the core language, Flask 2.x as the web framework, psycopg2 for PostgreSQL connectivity, and watchdog for event-driven file monitoring. The frontend is built with vanilla HTML, CSS, and JavaScript with Chart.js for dashboard visualizations.

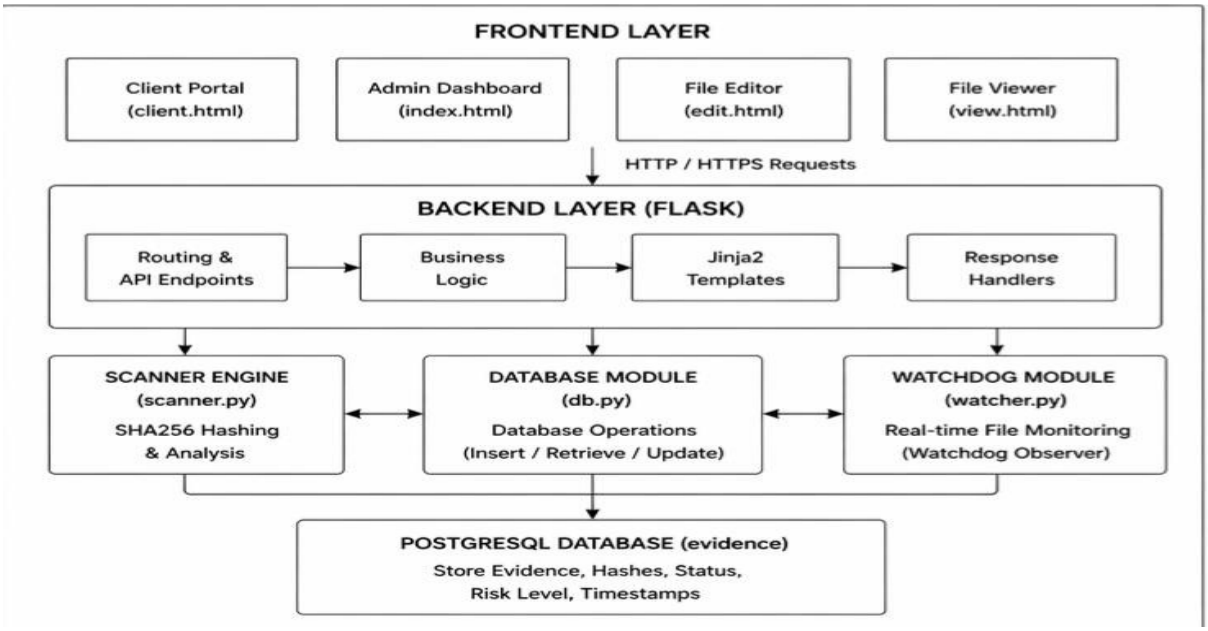

Figure 2.1: High-Level Architecture Diagram

2.2 Process Flow

The system operates through two parallel flows: a manual file upload flow and a real-time monitoring flow, both converging at the forensic scan engine. In the upload flow, a user submits a file via the client portal (POST /upload), where Flask validates and stores it in the scan_folder before triggering the scan process. In the monitoring flow, a watchdog observer continuously tracks the scan_folder and, upon detecting file changes, invokes a controlled scan function with a short delay to avoid duplicate processing. The forensic scan engine then analyzes all files by generating SHA256 hashes, comparing them with previous records stored in PostgreSQL, identifying file states (new, modified, unchanged, or deleted), assigning risk levels, and saving the results. These results are displayed on the admin dashboard, which fetches live data through multiple APIs and refreshes automatically at regular intervals. Additionally, users can export the latest scan data as a dynamically generated CSV file without storing it on disk.

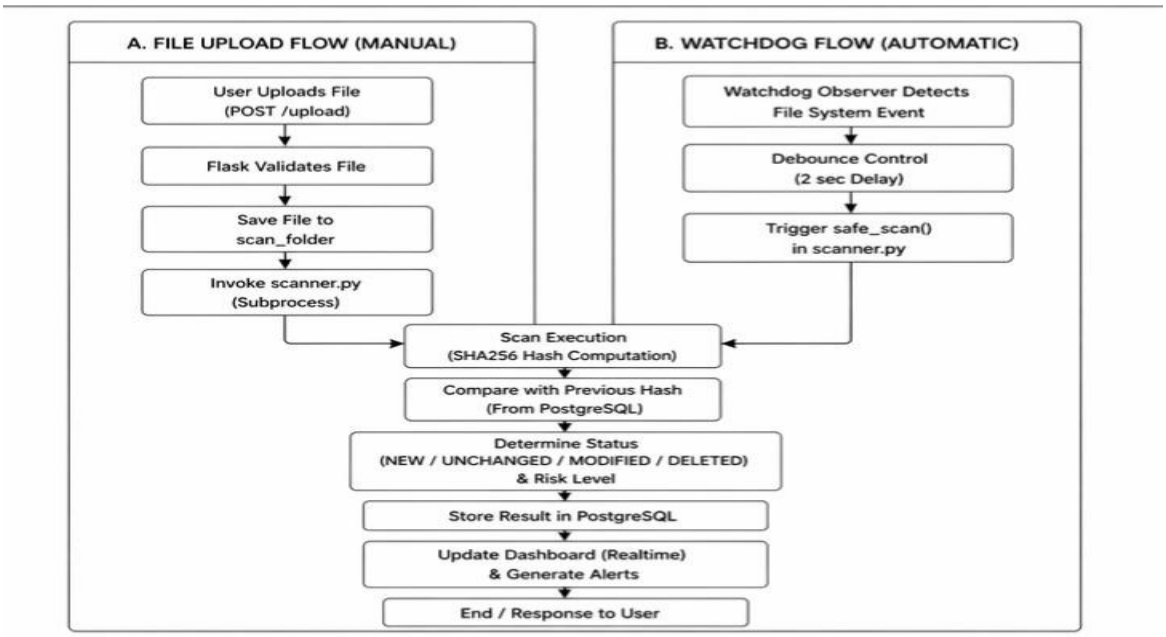

Figure 2.2: System Process Flow Diagram

2.3 Information Flow

The system manages data flow from user input to forensic output through multiple integrated stages. Files are either uploaded via the client portal or placed directly in the scan folder, where a monitoring service detects changes and forwards them to the scan engine. The processing component then computes SHA256 hashes, compares them with previous records stored in PostgreSQL, determines file status (new, modified, unchanged, or deleted), assigns risk levels, and prepares structured evidence data. This data is stored in the database with relevant metadata and timestamps. The backend, built with Flask, retrieves and serves this data through APIs, which provide summaries, detailed evidence, and historical trends. The admin dashboard visualizes this information using charts and tables with color-coded indicators for quick analysis. Additionally, the system supports exporting the latest scan results as a dynamically generated CSV file streamed directly to the user without saving it to disk.

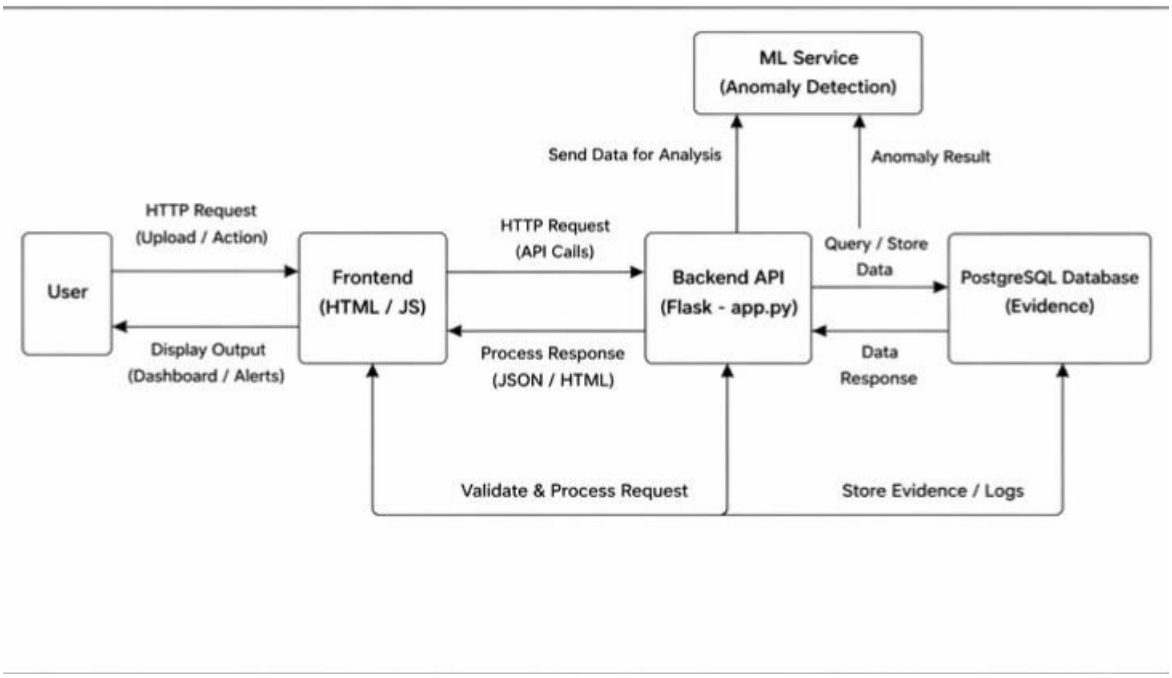

Figure 2.3: System Data Flow Diagram

2.4 Components Design

The system is structured into five interconnected components coordinated by a central controller. The main module, built using Flask, handles HTTP routing, user interactions, API responses, file operations, and triggers scanning processes while communicating with PostgreSQL for data access. The forensic engine processes files by scanning the directory, generating SHA256 hashes, comparing them with previous records, determining file status (new, modified, unchanged, or deleted), assigning risk levels, and storing results. A dedicated data layer manages all database operations, including inserting scan results and retrieving previous or current states. Real-time monitoring is handled by a separate event listener that watches file system changes and ensures controlled scan execution to avoid duplication. Finally, the frontend interface, built with template-based views, provides file upload functionality, a live-updating forensic dashboard with charts and logs, and additional features for file viewing, editing, and exporting results.

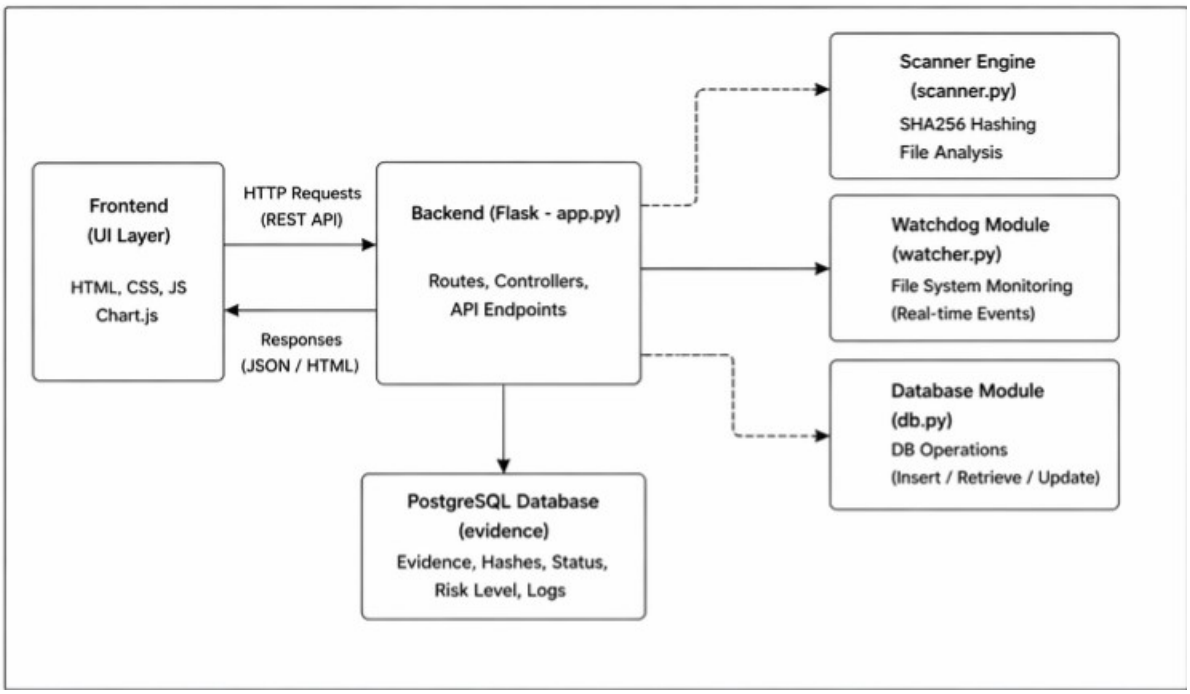

Figure 2.4: Component Design Diagram

2.5 Key Design Considerations

- **PostgreSQL as primary evidence store:** All scan results are persisted in PostgreSQL rather than CSV files. This ensures data is queryable, transactional, and reliable. CSV files are generated only on demand for export purposes.
- **Chunked SHA-256 hashing:** Files are processed using efficient chunk-based SHA-256 hashing, allowing the system to handle very large files (including those larger than 1 GB) without excessive memory usage.
- **Debounce control in file watcher:** A threading.Lock with a 2-second debounce threshold prevents duplicate scans caused by rapid filesystem events such as repeated saves or bulk uploads.
- **Subprocess isolation for scanner:** The scanner.py module is currently invoked using subprocess.run() to isolate the forensic engine from the Flask application process. A future refactor may replace this with direct method calls for improved efficiency.
- **Auto-refresh dashboard:** The frontend uses setInterval(loadDashboard, 3000) to refresh data every 3 seconds, enabling near real-time forensic monitoring without requiring manual page reloads.

- **Path traversal prevention:** All filenames are sanitized using Werkzeug's secure_filename() before any file system operations to prevent directory traversal or path injection attacks.
- **Risk-prioritized sorting:** Evidence records are ordered by severity: DELETED, MODIFIED, NEW, and UNCHANGED, ensuring that the most critical forensic events appear first.

2.6 API Catalogue

The system exposes the following API endpoints served by the Flask backend:

Endpoint	Method	Input	Description
GET /	GET	None	Serves the client file upload portal (client.html)
GET /admin	GET	None	Serves the forensics admin dashboard (index.html)
POST /upload	POST	multipart/form-data file field	Saves file to scan_folder, triggers scanner.py, redirects to /
GET /view/<filename>	GET	filename in URL	Displays file content in browser via view.html
GET /edit/<filename>	GET	filename in URL	Opens file in editor via edit.html
POST /save/<filename>	POST	form-data content field	Saves edited content, updates SHA256 and status in PostgreSQL
GET /download/<filename>	GET	filename in URL	Downloads file as attachment
GET /api/evidence	GET	None	Returns latest scan results as JSON, sorted by risk priority
GET /api/summary	GET	None	Returns file counts by status for dashboard stat cards

GET /api/history	GET	None	Returns scan history grouped by scan_time for trend chart
POST /api/scan	POST	None	Triggers manual scanner.py execution, returns {status: ok}
GET /api/export	GET	None	Streams forensics_evidence.csv generated dynamically from PostgreSQL

3. Data Design

3.1 Data Model

The system uses PostgreSQL as the sole persistent data store. All forensic evidence is stored in a single evidence table. No CSV or flat-file data store is used for persistence — CSV is generated on-demand only for export purposes.

The evidence table schema is designed to capture a complete forensic snapshot per file per scan session:

Column Name	Data Type	Example	Description
id	SERIAL PRIMARY KEY	1, 2, 3...	Auto-incrementing unique row identifier
file_name	TEXT	sales_data.txt	Base filename — display and lookup key
file_path	TEXT	scan_folder\\file.txt	Full relative path — used for hash comparison
sha256	TEXT	4f9c00caf2...	64-char hex SHA256 hash — tamper fingerprint

status	TEXT	MODIFIED	NEW | UNCHANGED | MODIFIED | DELETED
risk_level	TEXT	HIGH	CRITICAL | HIGH | MEDIUM | LOW
created_time	TIMESTAMP	2026-03-04 13:53:00	os.getctime() — inode change time (not reliable on Linux)
modified_time	TIMESTAMP	2026-04-21 10:54:00	os.getmtime() — primary tamper detection timestamp
scan_time	TIMESTAMP	2026-04-21 10:54:58	Timestamp grouping all files in one scan session

3.2 Data Access Mechanism

All database interactions are handled exclusively through db.py using psycopg2. The system uses a direct connection pattern — each function opens a connection, performs its operation, and closes it. The key data access functions are:

9. Bulk-inserts scan results using cursor.executemany(). Called at the end of every scan session in scanner.py.insert_data(rows):
10. Retrieves the previous scan's file_path and sha256 values for hash comparison. Identifies the previous scan_time using SELECT MAX(scan_time) WHERE scan_time < latest.get_previous_hashes():
11. Returns all rows from the latest scan session (WHERE scan_time = MAX(scan_time)), sorted by risk priority.get_current_state():

The Flask app.py also connects directly to PostgreSQL for the /api/summary, /api/history, and /api/export routes using the connect_db() function with the DB_CONFIG dictionary.ment.

3.3 Data Retention Policies

The system retains all scan results in PostgreSQL indefinitely, unless explicitly cleared by an administrator. Each scan session appends new records, ensuring that historical data is preserved to maintain a complete forensic audit trail.
Administrators can manage stored data using standard PostgreSQL commands:

- TRUNCATE TABLE evidence RESTART IDENTITY; — permanently removes all records and resets the primary key sequence.
- DELETE FROM evidence WHERE file_name = 'filename'; — removes records associated with a specific file.

The dashboard is designed to display only the most recent scan session using a query such as: WHERE scan_time = (SELECT MAX(scan_time) FROM evidence).

3.4 Data Migration

The current implementation uses a single flat `evidence` table without a formal migration strategy. As the system evolves, the following migration requirements should be considered:

- **Deduplication strategy (corrected constraint design):** Adding a `UNIQUE` constraint on `file_name` alone is not appropriate in a forensic context, as the same file may legitimately appear in multiple scan sessions.
  Instead, a composite constraint such as `(file_name, scan_time)` or `(file_name, file_hash)` should be used. This allows safe implementation of `ON CONFLICT DO UPDATE` for controlled deduplication without losing historical records.
- **Composite indexing for performance:** Introduce composite indexes on `(scan_time, status)` to improve query performance for dashboard operations, particularly as the `evidence` table grows over long monitoring periods.
- **Chained hash for integrity verification:** Add a `chained_hash` column to store the SHA-256 hash of the previous record in a defined sequence. This enables tamper-evident, blockchain-style verification of the evidence chain.
  A consistent ordering mechanism (e.g., by `scan_time` and primary key) must be enforced to ensure reliable hash computation.
- **Migration management framework:** Future schema changes should be handled using Alembic in combination with Flask-SQLAlchemy if the system transitions to an ORM-based architecture. This ensures version-controlled, repeatable, and reversible database migrations.

4. Interfaces

The system manages the following internal and external interfaces:

User Interface

A four-page web interface rendered by Flask using Jinja2 templates. The client portal (client.html) provides file upload functionality and a live file list with View, Edit, and Download actions per file. The admin dashboard (index.html) provides forensic monitoring with stat cards, pie chart (status distribution), bar chart (risk levels), line chart (scan history trend), evidence log table with colour-coded status badges, and an Export CSV button.

Internal Component Interfaces

- Via subprocess.run(['python', 'scanner.py']). Triggered on file upload and POST /api/scan.app.py → scanner.py:
- Direct Python module import. scanner.py calls insert_data() and get_previous_hashes() from db.py.scanner.py → db.py:
- Direct Python import. WatchHandler calls scan() from scanner.py on filesystem events with debounce control.watcher.py → scanner.py:
- Direct psycopg2 connection via connect_db() for all API route queries (/api/evidence, /api/summary, /api/history, /api/export).app.py → PostgreSQL:
- JavaScript fetch() calls to /api/summary, /api/evidence, and /api/history every 3 seconds. Chart.js renders JSON responses into visual charts.Frontend → Flask API:

External Interfaces

- Primary evidence store. Connected via psycopg2 on localhost:5432. Credentials stored in DB_CONFIG in app.py and db.py.PostgreSQL (forensics_db):
- Loaded from cdn.jsdelivr.net for dashboard chart rendering. No server-side dependency.Chart.js CDN:
- Used by watcher.py to observe filesystem events on scan_folder using the OS-native event notification API.Watchdog library:

5. State and Session Management

The Cloud Forensics system is primarily a stateless application at the request level. Flask processes each HTTP request independently without maintaining server-side session state between requests.

- The forensic state (which files exist, their hashes, their status) is persisted entirely in PostgreSQL, not in Flask session objects or in-memory variables.
- The watcher.py process maintains minimal state — only the last_run timestamp and a threading.Lock for debounce control. This state is process-local and resets on watcher restart.
- The admin dashboard maintains client-side state via JavaScript variables (pieChart, barChart, lineChart) for Chart.js instance tracking, and uses setInterval for auto-refresh timing.
- No authentication session management, cookies, or JWT tokens are implemented in the current version. The /admin route is publicly accessible. Flask-Login with session tokens is planned as a future security enhancement.
- No user-specific state is tracked. The system is designed as a single-operator local forensic tool. Multi-user support would require session management and role-based access control.

6. Caching

Version 1.0 of the system does not implement any explicit caching at any layer. This is intentional all scan results must always reflect the current state of the evidence database to ensure forensic accuracy. Cached or stale evidence data would undermine the integrity of the forensic record.

- No response caching is applied. Every /api/evidence, /api/summary, and /api/history request executes a fresh PostgreSQL query.Flask backend:
- No client-side caching is applied to API responses. fetch() calls are made fresh on every 3-second refresh cycle.Frontend JavaScript:
- HTML, CSS, and the Chart.js CDN script are cached by the browser using standard HTTP cache-control headers. This is acceptable as these assets are non-evidence data.Browser static assets:

In future versions, a short-lived in-memory cache (such as Python functools.lru_cache or TTLCache from the cachetools library could be applied to the /api/history endpoint, as historical scan data does not change between new scans. This would reduce database load on the history query during high-frequency dashboard refreshes.

7. Non-Functional Requirements

7.1 Security Aspects

The following security measures are implemented in the current version:

- All uploaded and accessed filenames are sanitized using Werkzeug's secure_filename() before any disk operation. This strips path separators (../), null bytes, and special characters, preventing escape from scan_folder.Path Traversal Prevention:
- scanner.py is called via subprocess.run() with a fixed argument list. No user-provided input is ever passed as a subprocess argument, preventing shell injection. shell=True is never used.Subprocess Security:
- All template variables are rendered through Jinja2, which applies automatic HTML escaping by default. User-supplied file content displayed in view.html is wrapped in <pre> tags with Jinja2 escaping.XSS Prevention:
- PostgreSQL evidence rows are intended as INSERT-only records — no DELETE or UPDATE on historical scan records. This preserves the forensic audit trail.Evidence Integrity:

The following security gaps are identified for future resolution:

Gap	Risk Level	Planned Fix
No authentication on /admin	HIGH	Flask-Login with session tokens and password hashing
No CSRF protection on POST routes	MEDIUM	Flask-WTF CSRFProtect on all POST routes
No rate limiting on /api/scan and /upload	MEDIUM	Flask-Limiter (10 scans/min, 20 uploads/hour)
DB credentials hardcoded in app.py and db.py	HIGH	Move to .env environment variables via python-dotenv
Subprocess call to scanner.py	LOW	Refactor to direct ForensicEngine.scan_directory() call
File upload without type validation	MEDIUM	Restrict to safe MIME types and extensions whitelist

7.2 Performance Aspects

- SHA256 is computed using efficient chunk-based hashing, allowing the scanner to handle files of any size — including those exceeding 1 GB — without loading the entire file into memory. Scan time increases linearly with file size.Chunked file hashing:
- The 2-second debounce threshold in watcher.py prevents redundant scans from rapid filesystem events such as editor auto-saves or bulk file operations.Debounce control in watcher:
- The 3-second setInterval refresh on the admin dashboard provides near-real-time evidence updates without WebSocket complexity. For high-frequency scan environments, this interval can be increased.Dashboard auto-refresh:
- The WHERE scan_time = (SELECT MAX(scan_time) FROM evidence) pattern efficiently retrieves only the latest scan snapshot. A future composite index on (scan_time, status) would further accelerate dashboard queries as the evidence table grows.PostgreSQL query optimization:
- The /api/export endpoint builds the CSV entirely in memory using io.StringIO and streams it directly — no disk I/O is required for export generation.CSV export performance:
- The current Flask development server (debug=True, single-threaded) is not suitable for production. Production deployment requires Gunicorn with multiple workers: gunicorn -w 4 -b 0.0.0.0:8000 app:app.Production scalability:

8. References

- Flask Official Documentation. https://flask.palletsprojects.com/
- psycopg2 Documentation. https://www.psycopg.org/docs/
- hashlib (SHA-256) Library Documentation. https://docs.python.org/3/library/hashlib.html
- Watchdog Documentation. https://python-watchdog.readthedocs.io/
- PostgreSQL Official Documentation. https://www.postgresql.org/docs/
- Werkzeug `secure_filename()` Utility. https://werkzeug.palletsprojects.com/
- Jinja2 Documentation. https://jinja.palletsprojects.com/
- Chart.js Documentation. https://www.chartjs.org/docs/latest/
- NIST https://csrc.nist.gov/publications/detail/sp/800-86/final
- OWASP https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTML